# The SMTLIB Plugin

1.0 Introduction....1
1.1 Commands....1
1.1.1 smt....1
1.1.2 smtexp....2
1.2 Design....3
1.2.1 Analysis Plugins....3
1.2.2 Analysis Commands....3
1.2.3 SMTLIB....3
1.2.4 VDM Expansion....4
1.2.5 Translation Visitors....5
1.2.6 Solvers....6
1.3 Issues....6
1.4 Future Directions....7
1.5 Source....7
1.6 References....7

## 1.0 Introduction

The SMTLIB plugin is experimental. It enables some VDMJ [1] Proof Obligations to be translated into the SMT-LIB [4] language and be submitted to an SMT solver [7]. The outcome of the solver will be interpreted as one of:

- PROVED: the obligation is true in all cases
- FAILED: a counterexample has been found by the solver
- MAYBE: neither of the above.

Note that these are similar to QuickCheck [3] outcomes. However obligations may be *proved* by this tool, rather than merely indicating that they are *probably provable*. This is because an SMT solver is considering all possible values, by symbolic reasoning, unlike QuickCheck.

The tool is experimental. It is currently only capable of processing obligations that concern simple types and expressions. But the hope is that the architecture of the plugin is sound, which will enable further development over time.

## 1.1 Commands

The plugin provides two commands, *smt* and *smtexp*.

### 1.1.1 smt

The *smt* command is available in both the VDMJ command line and the VDM-VSCode [2] Console.

```
> help smt
smt [-v][-<solver>] <PO#> [<PO#>...] – experimental SMT solver PO verification
>
```

The -v option (verbose) lists the SMT-LIB source that is generated for each PO considered. The second optional argument is the solver to use. By default, the command will use the Z3 solver [5], but an adapter for CVC5 [6] is also available. So the option -cvc5 would use that.

```
> pog
Generated 1 proof obligation:

Proof Obligation 1: (Unproved)
f: subtype obligation in 'DEFAULT' (test.vdm) at line 2:5
(forall a:nat &
  (a - 1) >= 0)

> smt 1
PO #1, FAILED in 0.005s
f: subtype obligation in 'DEFAULT' (test.vdm) at line 2:5
(forall a:nat &
  (a - 1) >= 0)
---- Counterexample:
a = 0
>
```

The tool is designed to allow arbitrary solvers to be added in future.

Note that the chosen solver must be installed separately, such that it can be executed on the default path with the name “z3” or “cvc5” etc. If the solver chosen is not installed, or there is no adapter for it, you will get the following error:

```
> smt -z4 1
Failed to execute solver: Unknown solver type: z4
>
```

### 1.1.2 smtexp

The *smtexp* command is only available on the VDMJ command line and is intended for debugging. Rather than trying to solve proof obligations, the command takes an arbitrary VDM boolean expression and attempts to solve that as though it was the body of a PO. This is much easier than trying to write a VDM spec that has obligations that contain an expression that you want to test. The command constructs a PO #0 containing the expression and then solves that.

Note that the command does not appear in the –help listing.

```
> smtexp
Usage: smtexp [-<solver>] <expression>
>
```

The command is roughly the same as generating a proof obligation with the expression passed, and then running the *smt* command with the -v verbose option. That is, it always displays the SMT-LIB output as well as the solver result.

```
> smtexp -z3 forall s:seq of nat, i:nat & s(i) = 0
; Proof Obligation #0
(set-option :produce-models true)
(set-info :smt-lib-version 2.6)
(set-logic ALL)
; Using solver z3
(push)
...
(check-sat)
(get-model)
(pop)
----
PO #0, FAILED in 0.023s
smtexp: theorem obligation
forall s:seq of nat, i:nat & s(i) = 0
---- Counterexample:
s = [2]
i = 0
>
```

## 1.2 Design

### 1.2.1 Analysis Plugins

Plugins are provided for the VDMJ command line and for the VDM-VSCode Console. These in turn contribute analysis commands which offer the *smt* and *smtexp* functionality. Both the VDMJ and VDM-VSCode commands call a common implementation class that does the work. This pattern keeps the different plugins and their commands separate from the main implementation.

Figure 1 shows the overall architecture. The orange parts are VDMJ components, and the blue parts are VDM-VSCode components. Both call the common grey components.

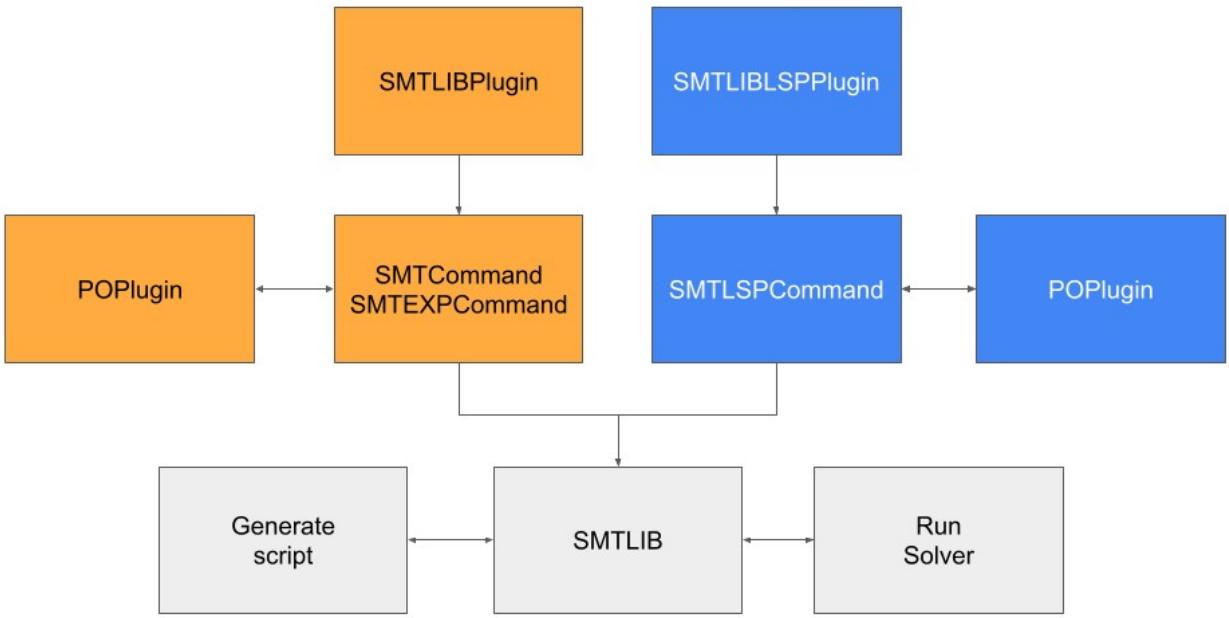



*Figure 1: VDMJ and VDM-VSCode plugin and command structure*

### 1.2.2 Analysis Commands

The analysis commands are responsible for parsing command line options, finding the proof obligations concerned, generating an SMT-LIB script that represents each obligation (or the expression passed to *smtexp*), invoking the chosen solver and displaying the result on the console.

Proof obligations are obtained from the *POPlugin.* The generation of SMT-LIB scripts and the execution of the solver is performed via the common SMTLIB class.

See Figure 1.

### 1.2.3 SMTLIB

The common SMTLIB class contains all of the functionality to generate SMT-LIB scripts and invoke a solver to analyse them.

The class is constructed with Environment and Solver objects, plus a verbose flag. The Environment allows the lookup of definitions within the current specification, which is needed

during SMT-LIB generation. The Solver object enables interaction with one particular solver. Implementations are provided for the Z3 and CVC5 solvers, but others can be added.

SMTLIB has two public methods:

- ***generate***. This is passed a proof obligation and produces a Script object, which is a sequence of SMT-LIB *Commands* (not to be confused with plugin commands). The bulk of the script is provided by the private *topLevelDefinitions* method.
- ***runSolver***. This is passed a Script and the proof obligation. It uses the Solver to create a child process and pass the script, then interprets the solver response and updates the PO status. This uses a private *parseModel* method (below).

There are two private methods:

- ***topLevelDefinitions***: This is passed a proof obligation and returns a Script, comprising SMT-LIB definitions of the PO expression and any additional functions that it requires (eg. if a precondition is called, it will be converted to a define-func). The method looks for the top level *forall* or *exists* expression and extracts its variable bindings, converting these into top level values for the solver to manipulate. The body of the expression is then translated using the visitors described below. In addition, the method creates additional top level definitions that are needed because the PO expression uses VDM operators which are not easily expressible in SMT-LIB.
- ***parseModel*:** This is passed a proof obligation and an open BufferedReader which is ready to read a counterexample (ie. following a “sat” response from the solver). The model is actually parsed using the Solver, since different solvers produce subtly different model formats. The *parseModel* updates the status, counterexample/witness and “explanation” of the PO.

### 1.2.4 VDM Expansion

The SMT-LIB language is not as rich as VDM-SL. For example, theories for integers and real numbers are distinct and there is no native support for natural numbers. All SMT-LIB functions are total, meaning VDM undefined values have to be handled explicitly.

To make the SMT-LIB translation easier, VDM-SL is first expanded into a simpler form which explicitly expresses things that SMT-LIB has to do, rather than using the implicit features of VDM. For example, natural numbers have to be treated as SMT-LIB “Int” sorts, qualified by an invariant explicitly:

```
forall x:nat & ...
forall x:int & inv_nat(x) => …
```

Here, the natural number is widened to an int, and an invariant is used to qualify the nat values that actually apply. These two VDM expressions are equivalent, but the second one can be directly converted to SMT-LIB.

Similarly, complex VDM pattern matching has to be simplified in the expansion:

```
forall mk_R(x, mk_Q(y, -)):R & ...
forall $R:R & let x = $R.fld1, y = $R.qfld.fld1 in ...
```

Here, the R and Q record patterns are used to disassemble the values being quantified over. This is turned into a simple quantification using an identifier pattern, $R, which is then used to extract the x and y values using let definitions ahead of the body.

Some expressions in VDM-SL are partial, meaning they have undefined values for some arguments. By contrast, all SMT-LIB expressions are total. So the partiality has to be propagated “up” the expression tree from such partial operators. For example:

```
forall s:seq of nat, i:nat1 & s(i) > s(i+1)

(forall s:seq of (int), i:int &
  (((forall $_i:int & (((0 < $_i) and ($_i <= (len s))) =>
    inv_nat(s($_i)))) and inv_nat1(i)) =>
      ((((0 < (i + 1)) and ((i + 1) <= (len s))) and
        ((0 < i) and (i <= (len s)))) and (s(i) > s((i + 1))))))
```

This example is much more complicated. Firstly, the sequence in the quantifier is a seq of nat, and the index is a nat1; these are both widened to int values. That creates the first implication, meaning that all of the sequence members and the index are nat and nat1. Then we have two partiality tests for the application of the two indexes to the sequence, saying that i and i+1 are in the range 1 to len s.

Note here that the quantifier constraints are implications (values outside are effectively ignored) whereas the partiality constraints are anded with the final predicate, meaning that if they are false, the whole expression is false.

SMT-LIB does not allow function definitions to include pre and postconditions, so these have to be explicitly expanded as well:

```
f([1,2]) > 5
((post_f([1, 2], f([1, 2])) and pre_f([1, 2])) and (f([1, 2]) > 5))
```

Here, we have a function that takes a sequence of ints and returns an int, and it has a precondition and a postcondition defined. The expansion explicitly applies the pre_f and post_f calls, the latter having to evaluate the function as well, to get the result. The calls to the function are translated to SMT-LIB which does not apply implicit pre/postconditions.

### 1.2.5 PO Translation Slicing

Rather than translating an entire specification and a given proof obligation to pass to the SMT solver, the tool only translates a slice of the specification, which only includes the definitions needed. This has advantages and disadvantages:

- Many POs are very simple and only reason about a few values and types. These can be translated into a very short SMT-LIB file and solvers can typically discharge them very quickly.
- The translation process is more complex, requiring a depth-first search of the required definitions. On the other hand, the ordering requirement for SMT-LIB (define before use) also requires a depth-first process.
- While many expressions cannot be handled, this approach means we can process as many POs as possible (if they only use simple expressions), while waiting for the tool to progress.

### 1.2.6 Translation Visitors

The bulk of the translation from VDMJ Expressions into SMT-LIB Expressions is performed by visitors which analyse the PO expression tree. These are the important visitors:

- ***ExpandExpression:***
  - This takes a TCExpression and returns another TCExpression, being the expanded form (see above). The two expressions are equivalent, but the expanded expression is directly translatable into SMT-LIB.
- ***ExpandType:***

- This takes a TCType and returns a QualifiedType, which is the equivalent needed in SMT-LIB and a qualifying expression, if needed. So nat would become int, inv_nat(i).
- The handling of these qualifiers can become quite complicated, when we are trying to qualify as few instances as possible, especially when qualifications percolate up from the leaves of the expression as well as down from the bindings that created them.
- Note that there are two distinct types of constraint. SMT-LIB sorts that are constrained to be VDM types are “given” qualifiers, whereas constraints that come from VDM partial operators (like *k in set dom m*) are “required”. So we can assume the given qualifiers, but if we violate the required constraints, that’s a counterexample.

- ***TypeToSMT***:
  - This converts TCTypes into SMT-LIB QualifiedSorts.
- ***ExpressionToSMT***:
  - This converts TCExpressions to Expressions (in the “ast” package).
  - Solvers have different levels of compliance with the SMT-LIB standard and some have non-standard (but very useful) extensions for some theories (like sets, seqs and maps). Therefore the translation process has to include the Solver object to take advantage of its capabilities.

### 1.2.7 Solvers

A number of SMT solvers are potentially available for this tool to use [7]. They all have different strengths and weaknesses, as well as varying compliance with the SMT-LIB Standard.

We want to be able to choose different solvers in the tool, so rather than hard coding support for particular ones, they are represented internally by an abstract Solver class, which gives the tool a standardised interface to work with. Solver instances are then created at startup (and cached) by looking for a Java resource called “smt.solvers”. This contains a list of Solver subclasses to use.

Solvers have the following methods:

- ***Init:*** A static method to use the smt.solvers resource to create Solver instances.
- ***getInstance:*** This static method returns the current instance of a given name, like “z3”. An overloaded method has no name and returns the default Solver (currently z3). So this implements a partial-singleton pattern, with one instance per name.
- ***getName:*** This method returns the simple name of the solver, like “z3” or “cvc5”. This is used to process command arguments that select a solver type.
- ***getProcess***: This returns a Java ProcessBuilder which will start the solver instance in such a way that it will accept an SMT-LIB script via its standard input. The builder should include any optimizing flags that are required.
- ***getVDMName***: This method is passed a counterexample binding from the solver and returns a VDM name. For example, a solver might return a model value in the form “(define-fun a () Real (/ 1 2))”. That would be converted into “a = 1/2” in VDM terms, so that it can sensibly be displayed as a counterexample. This translation is solver-specific.
- ***getVDMValue:*** this is passed a VDM value returned from the model, in SMT-LIB form. It attempts to turn the value into a VDM syntax. For example (as seq.empty Int) becomes [ ].
- ***getExplanation:*** This method returns an “explanation” for a PO, which includes the PO source as well as the name/value bindings of a witness or counterexample that has been found.

In addition to these, a Solver implements an interface called *VDMSolver*, which defines a set of methods for translating VDM operators whose equivalent is different in different solvers. These are then called by *ExpressionToSMT* rather than directly encoding the translation.

The “smt.solvers” resource is included with the plugin jar and contains the following entries:

```
smtlib.solvers.Z3Solver
smtlib.solvers.CVC5Solver
```

These are the two concrete subclasses of Solver which implement the Z3 and CVC5 adapters. But if you want to provide a further solver, package it into its own jar, add an “smt.solvers” resource listing the new class, and add the jar to the classpath. The tool’s initialization will instantiate the new solver as well as the standard ones, and you will be able to specify (say) “*smt -opensmt ...*”

## 1.3 Issues

In no particular order:

- The current implementation only handles very simple PO expressions, which implies very simple VDM specifications. Types are limited, as are expressions. If you attempt to process something it can’t handle, it will fail with a very basic explanation.
- The Z3 solver is much better at sets and seqs than CVC5, but it is not as good with maps. Z3 is the default for this reason.
- It is annoying that we have to use the very latest versions of the solvers, because although these are the most powerful, they also have the most non-standard features. That means that the Solver classes have to do more work.
- Solvers do typically have a lot of command line options for fine-tuning their behaviour. It may be that some of these would be useful for us. I did try a few at random, but there was no clear change in performance.
- There was an early high-level decision to use SMT-LIB source as the means to communicate with the solvers. It is widely supported and supposedly common. API bindings do exist for other languages in some solvers, but the standardization here is even looser than SMT-LIB.

## 1.4 Future Directions

Again, in no particular order:

- We should look at whether other language bindings are more powerful than SMT-LIB. We can consider simple “API” bindings, which effectively allow you to do what SMT-LIB does but via Java or Python etc. But we can also consider more radical translations, such as to Why3 or Boogie, and let them worry about the solvers, while we worry about how to interpret their output in VDM terms.
- But we should also be careful about the compromise between flexibility (to use multiple solvers) and complexity (that this introduces in the translation process and the tool environment and its maintenance).
- We can potentially analyse a PO expression and choose a solver based on what features are needed and which solvers are best at that. Similarly, multiple solvers could be invoked if the weaker ones return “unk” rather than a clear “sat” or “unsat”.
- From a teaching perspective, very simple specifications are quite normal and using SMT to genuinely prove POs for students might be a useful feature.

- It would be wise to keep an eye on the latest SMT solver releases and features, since their capabilities already vary considerably and new features are coming out all the time.

## 1.5 Source

The source code for the SMTLIB plugin is currently in the VDMJ repo, in the “smtlib” module.

If you would like early access, contact the VDMJ maintainer: nick.battle@gmail.com

## 1.6 References


[1] VDMJ, https://github.com/nickbattle/vdmj

[2] VDM-VSCode, https://github.com/overturetool/vdm-vscode

[3] QuickCheck, https://github.com/nickbattle/vdmj/wiki/Using-QuickCheck

[4] The SMT-LIB Standard: Version 2.6, http://www.SMT-LIB.org

[5] The Z3 theorem prover, https://github.com/z3prover/z3

[6] The CVC5 theorem prover, https://github.com/cvc5/cvc5

[7] SMT solvers, https://en.wikipedia.org/wiki/Satisfiability_modulo_theories